一、n-gram（传统一）

1.什么是 n-gram 模型

这种情况下的语言模型称为 *n* 元文法(*n*-gram)模型。

通常地，

- **当 n=1 时，即出现在第 i 位上的基元 w_i 独立于历史。**
  一元文法也被写为 uni-gram **或** monogram；
- **当 n=2 时，2-gram** (bi-gram) **被称为1阶马尔可夫链；**
- **当 n=3 时，3-gram**(tri-gram)**被称为2阶马尔可夫链，**
  依次类推。

n-gram 是一种**基于统计学习的语言模型**。它的基本思想是将文本里面的内容按照字节进行大小为 N 的滑动窗口操作。n-gram 语言模型基于这样的假设：每个词的出现只与前面 N-1 个词相关。具体来说，n-gram 首先在句首添加<BOS>、句尾添加<EOS>，然后使用长度为 n 的滑动窗口截取句段，生成集合。对于 n-gram，生成句子 s 的概率 p(s) 可以分解为（注意 w_0 和 w_m+1 的含义）：

$$p(s)=\prod_{i=1}^{m+1} p(w_i \mid w_{i-n+1}^{i-1})$$

其中，w_i^j 表示词序列 $w_i \dots w_j$，w_{i-n+1} 从 w_0 开始，
w_0 为 <BOS>，w_{m+1} 为 <EOS>。

2.给定一个句子，写出 1-gram，2-gram，3-gram

◆举例：

给定句子：John read a book
增加标记：<BOS> John read a book <EOS>

Unigram: <BOS>, John, read, a, book, <EOS>

Bigram: (<BOS>John), (John read), (read a),
(a book), (book <EOS>)

Trigram: (<BOS>John read), (John read a),
(read a book), (a book <EOS>)

3.四个句子作为训练集，计算第二问句子的 2-gram 概率（平滑和不平滑）

<BOS> John read a book <EOS>

基于2元文法的概率为：

$$p(\text{John read a book}) = p(\text{John}|\text{<BOS>}) \times p(\text{read}|\text{John}) \times p(\text{a}|\text{read}) \times p(\text{book}|\text{a}) \times p(\text{<EOS>}|\text{book})$$

其中条件概率可以通过训练集统计得到：

对于 ***n*-gram**，参数 $p(w_i \mid w_{i-n+1}^{i-1})$可由最大似然估计求得:

$$p(w_i \mid w_{i-n+1}^{i-1}) = f(w_i \mid w_{i-n+1}^{i-1}) = \frac{c(w_{i-n+1}^{i})}{\sum_{w_i} c(w_{i-n+1}^{i})} \quad \ldots(5\text{-}5)$$

其中，$\sum_{w_i} c(w_{i-n+1}^{i})$ 是历史串 w_{i-n+1}^{i-1} 在给定语料中出现的次数，即 $c(w_{i-n+1}^{i-1})$，不管 w_i 是什么。

$f(w_i \mid w_{i-n+1}^{i-1})$ 是在给定 w_{i-n+1}^{i-1} 的条件下 w_i 出现的相对频度，分子为 w_{i-n+1}^{i-1} 与 w_i 同现的次数。

具体算法:

<BOS>John read Moby Dick<EOS>
<BOS>Mary read a different book<EOS>
<BOS>She read a book by Cher<EOS>

$$p(John \mid <BOS>) = \frac{c(<BOS>\ John)}{\sum_w c(<BOS>\ w)} = \frac{1}{3}$$

$$p(read \mid John) = \frac{c(John\ \ read)}{\sum_w c(John\ \ w)} = \frac{1}{1}$$

$$p(a \mid read) = \frac{c(read\ \ a)}{\sum_w c(read\ \ w)} = \frac{2}{3}$$

$$p(book \mid a) = \frac{c(a\ \ book)}{\sum_w c(a\ \ w)} = \frac{1}{2}$$

$$p(<EOS> \mid book) = \frac{c(book\ \ <EOS>)}{\sum_w c(book\ \ w)} = \frac{1}{2}$$

$$p(John\ read\ a\ book) = \frac{1}{3} \times 1 \times \frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} \approx 0.06$$

<BOS>John read Moby Dick<EOS>
<BOS>Mary read a different book<EOS>
<BOS>She read a book by Cher<EOS>

$p(Cher\ read\ a\ book) = ?$

$$= p(Cher \mid <BOS>) \times p(read \mid Cher) \times p(a \mid read) \times p(book \mid a) \times p(<EOS> \mid book)$$

$$p(Cher \mid <BOS>) = \frac{c(<BOS>\ Cher)}{\sum_w c(<BOS>\ w)} = \frac{0}{3}$$

$$p(read \mid Cher) = \frac{c(Cher\ \ read)}{\sum_w c(Cher\ \ w)} = \frac{0}{1}$$

于是，$p(Cher\ read\ a\ book) = 0$

平滑是什么?

问题:

数据匮乏(稀疏) (*Sparse Data*) 引起零概率问题，如何解决?

数据平滑(data smoothing)

困惑度了解即可:

➢ **回顾—困惑度的定义：**

对于一个平滑的 ***n*-gram**，其概率为 $p(w_i \mid w_{i-n+1}^{i-1})$，可以计算句子的概率：

$$p(s)=\prod_{i=1}^{m+1} p(w_i \mid w_{i-n+1}^{i-1})$$

假定测试语料 T 由 l_T 个句子构成 $(t_1,...t_{l_T})$，则整个测试集的概率为：

$$p(T)=\prod_{i=1}^{l_T} p(t_i)$$

模型 $p(w_i \mid w_{i-n+1}^{i-1})$ 对于测试语料的交叉熵：

$$H_p(T)=-\frac{1}{W_T}\log_2 p(T)$$

其中，W_T 是测试文本 T 的词数。

模型 p 的困惑度 $PP_p(T)$ 定义为：$PP_p(T)=2^{H_p(T)}$

***n*-gram 对于英语文本的困惑度范围一般为 50～1000，对应于交叉熵范围为 6～10 bits/word。**

+1 平滑方法：

加1法(Additive smoothing)

基本思想：每一种情况出现的次数加1。

例如，对于 ***uni*-gram**，设 w_1, w_2, w_3 三个词，概率分别为：**1/3, 0, 2/3**，加1后情况？

2/6，1/6，3/6

2-gram 平滑方法（重要）

对于*2*-gram 有：

$$p(w_i \mid w_{i-1})=\frac{1+c(w_{i-1}w_i)}{\sum_{w_i}[1+c(w_{i-1}w_i)]}$$

$$=\frac{1+c(w_{i-1}w_i)}{|V|+\sum_{w_i} c(w_{i-1}w_i)}$$

其中，V 为被考虑语料的词汇量（全部可能的基元数）。

实际上 V = 语料库中不同词的数量（不包括<BOS>和<EOS>）

如果是 n-gram 那么 V = 语料库中不同词的数量^(n-1)

在前面 3 个句子的例子中，

$p(Cher\ read\ a\ book)=p(Cher|<BOS>)\times p(read|Cher)\times p(a|read)\times p(book|a)\times p(<EOS>|book)$

<BOS>John read Moby Dick<EOS>
<BOS>Mary read a different book<EOS>
<BOS>She read a book by Cher<EOS>

原来：

$p(Cher|<BOS>)=0/3$

$p(read|Cher)=0/1$

$p(a|read)=2/3$

$p(book|a)=1/2$

$p(<EOS>|book)=1/2$

词汇量：$|V|=11$

<BOS>John read Moby Dick<EOS>
<BOS>Mary read a different book<EOS>
<BOS>She read a book by Cher<EOS>

平滑以后：

$p(Cher|<BOS>)=(0+1)/(11+3)=1/14$

$p(read|Cher)=(0+1)/(11+1)=1/12$

$p(a|read)=(1+2)/(11+3)=3/14$

$p(book|a)=(1+1)/(11+2)=2/13$

$p(<EOS>|book)=(1+1)/(11+2)=2/13$

$$p(Cher\ read\ a\ book)=\frac{1}{14}\times\frac{1}{12}\times\frac{3}{14}\times\frac{2}{13}\times\frac{2}{13}\approx 0.00003$$

(2) 减值法/折扣法(Discounting)

基本思想：修改训练样本中事件的实际计数，使样本中(实际出现的)不同事件的概率之和小于1，剩余的概率量分配给未见概率。

- **四种减值法的比较**
  - **Good-Turing 法：**对非0事件按公式削减出现的次数，节留出来的概率均分给0概率事件。
  - **Katz 后退法：**对非0事件按Good-Turing法计算减值，节留出来的概率按低阶分布分给0概率事件。
  - **绝对减值法：**对非0事件无条件削减某一固定的出现次数值，节留出来的概率均分给0概率事件。
  - **线性减值法：**对非0事件根据出现次数按比例削减次数值，节留出来的概率均分给0概率事件。

(3) 删除插值法 (Deleted interpolation)

基本思想：用低阶语法估计高阶语法，即当 3-gram 的值不能从训练数据中准确估计时，用 2-gram 来替代，同样，当 2-gram 的值不能从训练语料中准确估计时，可以用 1-gram 的值来代替。插值公式：

$$p(w_3 \mid w_1 w_2) = \lambda_3 p'(w_3 \mid w_1 w_2) + \lambda_2 p'(w_3 \mid w_2) + \lambda_1 p'(w_3) \quad \dots (5\text{-}13)$$

其中，$\lambda_1 + \lambda_2 + \lambda_3 = 1$

简单加权

二、分词（传统二）

1.常见的中文分词方式

基于词表的分词方法：

正向最大匹配法(forward maximum matching method, FMM)

逆向最大匹配法(backward maximum matching method, BMM)

N-最短路径方法

基于统计模型的分词方法：

基于 N-gram 语言模型的分词方法

基于序列标注的分词方法：

统计机器学习方法：

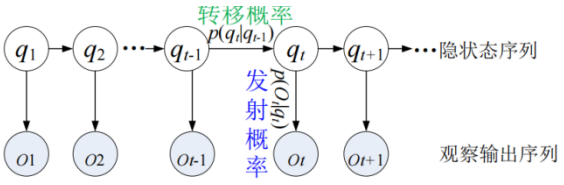
基于 HMM 的分词方法

基于 CRF 的分词方法

基于**词感知机**的分词方法

基于深度学习的端到端的分词方法（由字构词 (基于字标注)的分词方法）

2.HMM 有哪些组成部分并解释



HMM 图解

◆隐马尔可夫模型 (Hidden Markov Model, HMM)

创建于20世纪70年代，美国数学家鲍姆(Leonard E. Baum)等人提出。

描写：该模型是一个双重随机过程，我们不知道具体的状态序列，只知道状态转移的概率，即模型的状态转换过程是不可观察的（隐蔽的），而可观察事件的随机过程是隐蔽状态转换过程的随机函数。

HMM 的组成：从 1~5 都要写

◆HMM 的组成

1. 模型中的状态数为 N (袋子的数量)
2. 从每一个状态可能输出的不同的符号数 M (不同颜色球的数目)

3. 状态转移概率矩阵 $A=a_{ij}$ (a_{ij} 为实验员从一只袋子(状态 s_i) 转向另一只袋子(状态 s_j) 的概率)。其中，

$$\begin{cases} a_{ij} = p(q_{t+1} = s_j \mid q_t = s_i), \quad 1 \le i, j \le N \\ a_{ij} \ge 0 \\ \sum_{j=1}^{N} a_{ij} = 1 \end{cases} \quad \dots (6.6)$$

4. 从状态 s_j 观察到某一特定符号 v_k 的概率分布矩阵为：

$B=b_j(k)$ 状态发射概率矩阵 B(NxM 矩阵)

其中，$b_j(k)$为 实验员从第 j 个袋子中取出第 k 种颜色的球的概率。那么，

$$\begin{cases} b_j(k) = p(o_t = v_k \mid q_t = s_j), \quad 1 \le j \le N, \quad 1 \le k \le M \\ b_j(k) \ge 0 \\ \sum_{k=1}^{M} b_j(k) = 1 \end{cases} \quad \dots (6.7)$$

5. 初始状态的概率分布为：$\pi = \pi_i$，其中，

$$\begin{cases} \pi_i = p(q_1 = s_i), \quad 1 \le i \le N \\ \pi_i \ge 0 \\ \sum_{i=1}^{N} \pi_i = 1 \end{cases} \quad \dots (6.8)$$

为了方便，一般将 HMM 记为：$\mu = (A, B, \pi)$

或者 $\mu = (S, O, A, B, \pi)$ 用以指出模型的参数集合。

HMM 工作过程：

◆给定HMM求观察序列

给定模型 $\mu = (A, B, \pi)$，产生观察序列 $O = o_1 o_2 \dots o_T$：

(1) 令 $t = 1$;
(2) 根据初始状态分布 $\pi = \pi_i$ 选择初始状态 $q_1 = s_i$;
(3) 根据状态 S_i 的输出概率分布 $b_i(k)$, 输出 $o_t = v_k$;
(4) 根据状态转移概率 a_{ij}，转移到新状态 $q_{t+1} = s_j$;
(5) $t = t+1$, 如果 $t < T$, 重复步骤 (3) (4), 否则结束。

迭代：“先发射再转移”

3.HMM 如何实现分词和词性标注，使用公式说明

思路：

如果把汉语自动分词结果作为观察序列 $O = O_1 O_2 \dots O_T$，

那么，要求解的是：$\hat{O} = \arg\max_O p(O \mid \mu)$ 。

对于词性标注而言，则需求解：$\hat{Q} = \arg\max_Q p(Q \mid O, \mu)$ 。

(1)估计HMM模型$\mu = (A, B, \pi)$ 的参数；(Train)
(2)对于任意给定的一个输入句子及其可能的输出序列O，求找所有可能的O中使概率$p(O \mid \mu)$最大的解；(分词)
(3)快速地选择“最优”的状态序列Q(词性序列)，使其最好地解释观察序列$p(Q \mid O, \mu)$ 。(词性标注)

先求分词划分（观察状态）、再求词性（隐状态）

Train（求出\mu）->分词（dp->max 观察序列）->求出来的隐状态序列 O（词性序列）

公式说明：使用语料库进行训练时

$$\bar{\pi}_{\mathrm{pos}_i} = \frac{\mathrm{POS}_i\text{出现在句首的次数}}{\text{所有句首的个数}}$$

$$\bar{a}_{ij} = \frac{\text{从词类}\mathrm{POS}_i\text{转移到}\mathrm{POS}_j\text{的次数}}{\text{所有从状态}\mathrm{POS}_i\text{转移到另一}\mathrm{POS}(\text{包括}\mathrm{POS}_j)\text{的总数}}$$

$$\bar{b}_j(k) = \frac{\text{从状态}\mathrm{POS}_j\text{输出词汇}w_k\text{的次数}}{\text{状态}\mathrm{POS}_j\text{出现的总次数}}$$

*4.语言模型如何处理中文分词（可举例说明）

采用基于语言模型的分词方法

➢ **方法描述：**

设对于待切分的句子$S = z_1 z_2 \ldots z_m$，$W = w_1 w_2 \ldots w_k$ $(1 \le k \le n)$ 是一种可能的切分。那么，

$$\begin{aligned}\hat{W} &= \arg\max_W p(W \mid S) \\ &= \arg\max_W p(W) \times p(S \mid W) \\ &\cong \arg\max_W p(W)\end{aligned}$$

最基本的做法是以词为独立的统计基元，但效果不佳。

对于中文的句子，其中的字以不同的方式组合到一起成为词。对于一个给定的句子 s 有多种切分方式。应用 n-gram 模型进行分词的过程主要包括两步：

1、将句子的所有划分方式罗列出来；

2、根据句子的不同划分，使用 n-gram 模型来计算该句子的概率；

3、找出概率最大的划分方式即为 n-gram 模型的分词结果。

5.5 语言模型应用举例

进一步做如下约定，把一个可能的词序列 W 转换成词类序列 $C = c_1 c_2 \cdots c_N$，即：

- **专有名词：人名PN、地名LN、机构名ON分别作为一类；**
- **实体名词中的日期 dat、时间tim、百分数per、货币mon 等作为一类；**
- **对词法派生词MW和词表词LW，每个词单独作为一类。**

词序列变为类序列：

dat/ tim/ PN/ 在/ LN/ 发表/ 讲话/ ，/ 决定/ 从/ tim/ 起/ ON/ 将/ 大/ 规模/ 招收/ 研发/ 人员/ ，/ 其中/ ，/ per/ 将/ 从/ 山东/ 大学/ 所/ 培养/ 的/ 应届/ 博士/ 或/ 硕士/ 毕业生/ 中/ 选拔/ ，/ 年薪/ 不/ 低于/ mon/ 。

那么， $\hat{C} = \arg\max_C p(C|S)$

$= \arg\max_C p(C) \times p(S|C)$ … (5-22)

语言模型　　生成模型

$p(C)$ 可采用三元语法：

$$p(C) = p(c_1) \times p(c_2|c_1) \prod_{i=3}^{N} p(c_i|c_{i-2}c_{i-1}) \quad \text{… (5-23)}$$

$$p(c_i|c_{i-2}c_{i-1}) = \frac{count(c_{i-2}c_{i-1}c_i)}{count(c_{i-2}c_{i-1})} \quad \text{… (5-24)}$$

语言模型产生词类 C 的概率 乘 根据词类 C 生成原句的概率

生成模型在满足独立性假设的条件下，可近似为：

$$p(S|C) \approx \prod_{i=1}^{N} p(s_i|c_i) \quad \text{… (5-25)}$$

该公式的含意是，任意一个词类 c_i 生成汉字串 s_i 的概率只与自身有关，而与其上下文无关。

例如，如果“**教授**”是词表里的词，那么

$p(s_i$=**教授**$|c_i$=LW)=1，否则，$p(s_i|c_i) = 0$。

◆分词与词性标注一体化方法

假设句子***S*** 是由单词串组成：

$$W = w_1 w_2 \cdots w_n \quad (n \geq 1)$$

单词 $w_i (1 \leq i \leq n)$ 的词性标注为 t_i，即句子***S*** 相应的词性标注符号序列可表达为：

$$T = t_1 t_2 \cdots t_n$$

那么，分词与词性标注的任务就是要在***S*** 所对应的各种切分和标注形式中，寻找 T 和 W 的联合概率 $p(W, T)$为最优的词切分和标注组合。

有三种思路：基于词性/单词的三元统计模型、分词词性一体化模型

(1)基于词性的三元统计模型：

$$p(W,T) = p(W|T) \times p(T) \approx \prod_{i=1}^{n} p(w_i|t_i) \times p(t_i|t_{i-1}, t_{i-2}) \quad \text{…(5-26)}$$

其中，$p(W|T)$ 称为生成模型，$p(w_i|t_i)$表示在整个标注语料中，在词性 t_i 的条件下，单词 w_i 出现的概率。

$p(T)$ 为基于词性的语言模型，采用三元文法，

当 i=1时，取$p(t_1)$；当 i=2 时，取 $p(t_2|t_1)$ 。

(2)基于单词的三元统计模型：

$$p(W,T) = p(T|W) \times p(W) \approx \prod_{i=1}^{n} p(t_i|w_i) \times p(w_i|w_{i-1}, w_{i-2}) \quad \text{…(5-27)}$$

其中，$p(t_i|w_i)$ 反映的是每个词对应词性符号的概率。

$p(w_i|w_{i-1}, w_{i-2})$ 是普通的三元语言模型，当 i=1时，取 $p(w_1)$；当 i=2 时，取 $p(w_2|w_1)$ 。

(3)分词与词性标注一体化模型：

$$p^*(W,T)=\alpha\prod_{i=1}^{n}p(w_i|t_i)\times p(t_i|t_{i-1},t_{i-2})+\beta\prod_{i=1}^{n}p(t_i|w_i)\times p(w_i|w_{i-1},w_{i-2}) \quad \ldots(5\text{-}28)$$

这种综合模型的指导思想是希望通过调整参数α和β的值来确定两个子模型在整个分词与词性标注过程中所发挥作用的比重，从而获得分词与词性标注的整体最优。(当i分别为1和2时的语言模型分别取一元文法概率和二元文法概率，在下面的两页中同样。)

一体化模型：前两个模型简单加权

$p(t_i|w_i)$对分词无帮助，且在分词确定后对词性标注又会增添偏差。

因此，可以仅取(5-27)中的语言模型部分，而舍弃词性标注部分，并令(5-28)中的$\alpha=1$，仅保留加权系统β，于是，(5-28)式成为：

$$\hat{p}(W,T)=\prod_{i=1}^{n}p(w_i|t_i)\times p(t_i|t_{i-1},t_{i-2})+\beta\prod_{i=1}^{n}p(w_i|w_{i-1},w_{i-2}) \quad \ldots(5\text{-}29)$$

答题思路：

1.方法描述：

设对于待切分的句子$S=z_1z_2\ldots z_m$，$W=w_1w_2\ldots w_k$（$1\le k\le n$）是一种可能的切分。那么，

$$\begin{aligned}\hat{W}&=\arg\max_W p(W|S)\\&=\arg\max_W p(W)\times p(S|W)\\&\cong\arg\max_W p(W)\end{aligned}$$

最基本的做法是以词为独立的统计基元，但效果不佳。

2.可以引入类序列的概念

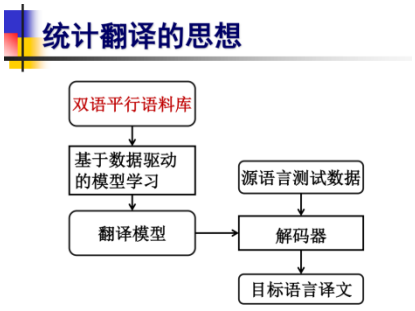
词类序列 $C=c_1c_2\cdots c_N$ ，即：

- 专有名词：人名PN、地名LN、机构名ON分别作为一类；
- 实体名词中的日期 dat、时间tim、百分数per、货币mon 等作为一类；
- 对词法派生词MW和词表词LW，每个词单独作为一类。

3. 同时处理分词和标注有三种思路：基于词性 和 基于单词 和 单词词性一体化

三、机器翻译（传统三）

1.画出统计机器翻译图示，通过概率公式说明翻译过程



原理：朴素贝叶斯

源语言句子： $S = s_1^m = s_1 s_2 \cdots s_m$

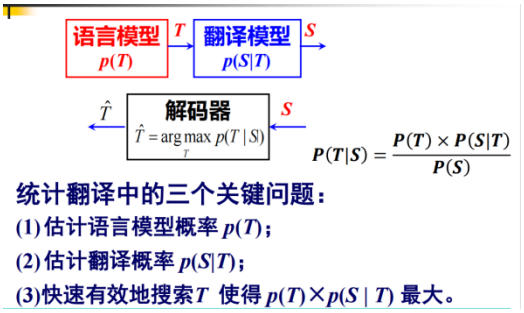
目标语言句子： $T = t_1^l = t_1 t_2 \cdots t_l$

贝叶斯公式： $P(T|S) = \dfrac{P(T) \times P(S|T)}{P(S)}$

$$T' = \arg\max_T P(T) \times P(S|T)$$

语言模型 Language model, LM

翻译模型 Translation model, TM



统计翻译中的三个关键问题：

(1) 估计语言模型概率 $p(T)$；

(2) 估计翻译概率 $p(S|T)$；

(3)快速有效地搜索 T 使得 $p(T) \times p(S|T)$ 最大。

◆估计语言模型概率 $p(T)$

给定句子： $T = t_1^l = t_1 t_2 \cdots t_l$

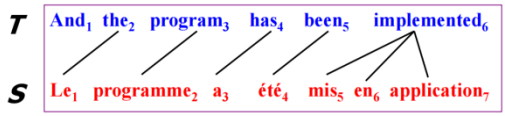
句子概率： $P(T) = P(t_1)P(t_2|t_1) \cdots P(t_l|t_1 t_2 \cdots t_{l-1})$

***n*-gram 问题，不再赘述。**

◆翻译概率 $p(S|T)$ 的计算

关键问题是怎样定义目标语言句子中的词与源语言句子中的词之间的对应关系。

假设英语与法语的翻译对：



不妨，我们用 $\mathcal{A}(S,T)$ 表示源语言句子 S 与目标语言句子 T 之间所有对位关系的集合。在目标语言句子 T 的长度（单词的个数）为 l，源语言句子 S 的长度为 m 的情况下，T 和 S 的单词之间有 $2^{l \times m}$ 种不同的对应关系。

$$|\mathcal{A}(S,T)| = 2^{l \times m} \qquad A(S,T) \in \mathcal{A}(S,T)$$

用来刻画这些对应关系 $A(S,T)$ 的模型叫做**对位模型**（alignment model）。

将对位模型 A 视为隐含变量，则：

$$P(S|T) = \sum_A P(S,A|T)$$

按照约定，源语言句子 $S = s_1^m = s_1 s_2 \cdots s_m$ 有 m 个单词

目标语言句子 $T = t_1^l = t_1 t_2 \cdots t_l$ 有 l 个单词

每一种对位序列表示成：

$$A = a_1^m = a_1 a_2 \cdots a_m \qquad a_j \in [0, 1, \cdots, l]$$

◆翻译概率 $P(S|T)$ 的计算

$$P(S|T)=\sum_{A}P(S,A|T)$$

$$P(S,A|T) \quad ?$$

$$P(S,A|T)=P(m|T)\times P(A|T,m)\times P(S|T,A,m)$$

对位模型　　词汇翻译模型

先以概率$P(m|T)$选择一个长度m，然后以概率$P(A|T,m)$生成对位序列A，然后以概率$P(S|T,A,m)$生成源序列S

11.2.1 统计翻译基本原理

$$P(S,A|T)=p(m|T)\times P(A|T,m)\times P(S|T,A,m)$$

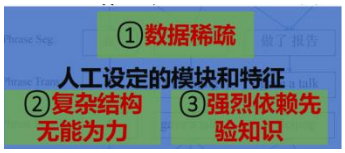
$$=p(m|T)\prod_{j=1}^{m}p\left(a_j|a_1^{j-1},s_1^{j-1},m,T\right)\times p\left(s_j|a_1^{j},s_1^{j-1},m,T\right)$$

实际上，$p(S,A|T)$ 可以写成多种形式的条件概率的乘积，上式只是其中的一种。在上式的基础上，IBM 的研究人员通过采用不同的假设条件得到了5个翻译模型，分别称作 IBM 翻译模型1、2、3、4 和 5。

*优点：



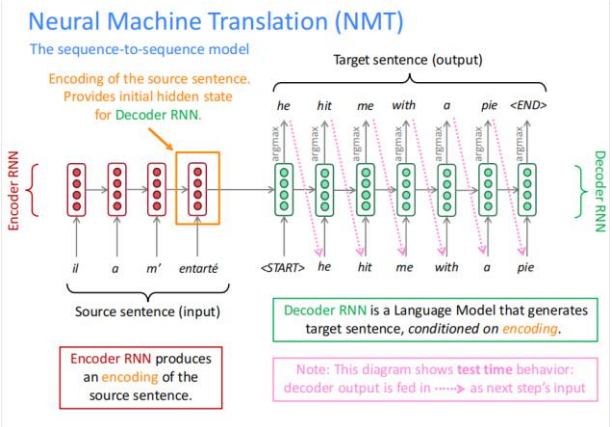
缺点：



2.画出基本的神经机器翻译(NMT)模型图示，介绍翻译过程

神经机器翻译是一种序列到序列模型

编码器：RNN 生成一个源句的编码。为解码器 RNN 提供初始 hidden state。
解码器：采用 RNN，是一种基于编码条件生成目标句子的语言模型。



从序列到序列的模型是条件语言模型的一个例子。

是语言模型，因为解码器正在预测目标句子 y 的下一个单词

这是有条件的，因为它的预测也以源句 x 为条件

Neural Machine Translation (NMT)

- The sequence-to-sequence model is an example of a **Conditional Language Model**.
  - **Language Model** because the decoder is predicting the next word of the target sentence *y*
  - **Conditional** because its predictions are *also* conditioned on the source sentence *x*
- NMT directly calculates $P(y|x)$:

$$P(y|x) = P(y_1|x)\,P(y_2|y_1,x)\,P(y_3|y_1,y_2,x)\ldots\underbrace{P(y_T|y_1,\ldots,y_{T-1},x)}$$

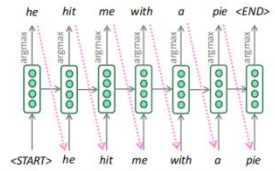
Probability of next target word, given target words so far and source sentence *x*

- **Question**: How to train a NMT system?
- **Answer**: Get a big parallel corpus…

使用贪心每次选择最大概率的单词

Greedy decoding

- We saw how to generate (or "decode") the target sentence by taking argmax on each step of the decoder



- This is greedy decoding (take most probable word on each step)
- **Problems with this method?**

存在缺点，使用 beam search

Beam search decoding

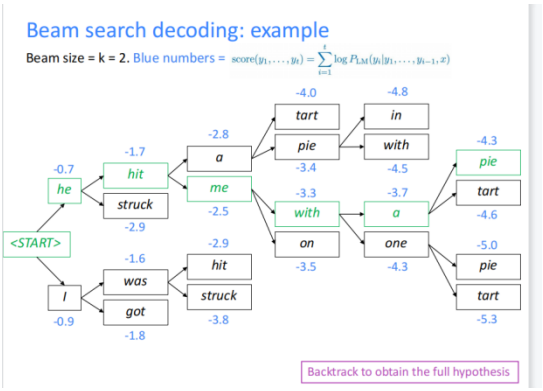
- Core idea: On each step of decoder, keep track of the *k* most probable partial translations (which we call *hypotheses*)
  - *k* is the beam size (in practice around 5 to 10)
- A hypothesis $y_1,\ldots,y_t$ has a score which is its log probability:

$$\text{score}(y_1,\ldots,y_t) = \log P_{\text{LM}}(y_1,\ldots,y_t|x) = \sum_{i=1}^{t}\log P_{\text{LM}}(y_i|y_1,\ldots,y_{i-1},x)$$

  - Scores are all negative, and higher score is better
  - We search for high-scoring hypotheses, tracking top *k* on each step
- Beam search is not guaranteed to find optimal solution
- But much more efficient than exhaustive search!

过程图示：

每次产生 k^2 个待选单词，从 k^2 中，选择 k 个概率最大的



Usually we continue beam search until:

- We reach timestep *T* (where *T* is some pre-defined cutoff), or
- We have at least *n* completed hypotheses (where *n* is pre-defined cutoff)

NMT 优点?

更好的表现：

更流畅、**更好**地**使用上下文**及**短语相似性**

端到端的优化，直接训练不需要拆分

不需要特征工程：需要更少的人力工作，**对所有语言适用**

四、情感分析（传统四）

介绍情感分析的关键技术并解释

◆ 情感分析研究观点挖掘、倾向性分析等

- ***观点挖掘与倾向性分析：*** 从海量数据中挖掘观点信息，并分析观点信息的倾向性
  - 非结构化→结构化

情感分析发展七项关键技术

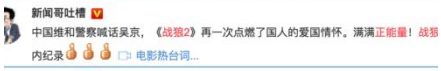
- 情感分类
- 情感元素抽取
- 跨领域情感分析
- 个性化情感分析
- 隐式情感分析
- 情感原因发现
- 情感生成

分抽跨个隐原生

情感分类

情感倾向性分类任务

- 输入：句子/篇章
- 输出：句子/篇章的褒贬极性及其强度



情绪分类任务

- 输入：句子/篇章
- 输出：句子/篇章情绪类别（如：喜怒悲恐惊等）



- 基于传统机器学习方法的情感分类
- 基于深度学习方法的情感分类
- 面向评价对象的情感分类

情感元素抽取

- 情感词抽取/表示/词典
  - 褒义：好，贬义：坏
- 评价对象抽取
  - 评价的实体
- 评价搭配抽取
  - <评价对象，情感词>

跨领域情感分析

- 从源领域到目标领域进行模型的迁移
- 目的
  - 标注少量或不标注目标领域的语料，利用源领域的语料在目标领域达到较好的性能

个性化情感分析

在情感分析中加入个性化的元素

情感分析的展示变得独特、另类、拥有自己特质的需要，独具一格

- 基于用户用词习惯的方法
  - 不同用户和群体情感倾向具有差异性
    - 由于用户群体立场存在差异，不同的用户群体往往对同一话题的情感倾向不同
    - 不同用户群体表达相同情感时，用词风格不尽相同
- 基于认知理论的方法
- 基于网络结构的方法

隐式情感分析

社会媒体中文本情感表达方式复杂

- 多数没有显式情感词
- 多使用语言修辞表达或事实性陈述

- 事实型隐式情感分析
- 修辞型隐式情感分析

情感原因发现

- 基于文本的情感原因发现
- 基于个体立场的情感原因发现
- 基于群体立场的情感原因发现

- 情感原因通常是由个体、所在群体及内容共同作用产生的

情感生成

- 评论文本生成
- 情绪对话生成

观点挖掘与倾向性分析相关任务

观点及倾向性识别

- 情感识别（Sentiment Identification）

观点要素抽取

- 观点属性抽取（Opinion Attribute Extraction）
- 观点摘要（Opinion Summarization）

观点检索

典型方法

- 情感识别
- 观点挖掘
- 观点检索
- 资源和评测

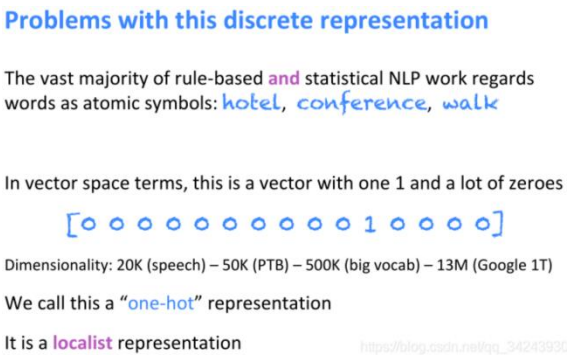
情感识别：词级别、句子级别、文档级别
观点挖掘：观点对象抽取、观点持有者抽取
观点检索：从海量文本中根据查询找到观点信息，找到主题相关度得分与观点倾向性得分的折中
资源：词典、语料　评测：相关会议

五、词向量（深度一）

1.离散化表示（one-hot）的缺点

Problems with this discrete representation

- Great as a resource but missing nuances, e.g., **synonyms**:
  - adept, expert, good, practiced, proficient, skillful?
- Missing new words (impossible to keep up to date): wicked, badass, nifty, crack, ace, wizard, genius, ninja
- Subjective
- Requires human labor to create and adapt
- Hard to compute accurate word similarity →



1.同义词数量众多，离散化表示不能体现同义词直接的相似性、很难计算精确的词间相似度
2. 需要人力制作，难以添加新词（维度更新!）
3. one-hot 每个词都需要一维导致向量长度太长
4.主观的

2.介绍两种词向量

One-hot 表示

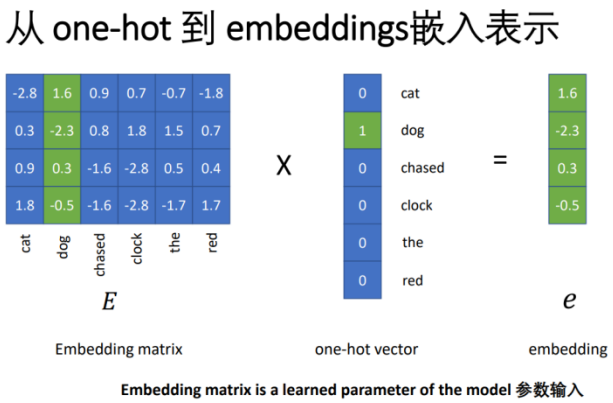
对单词进行编码，过程:
首先对 n 个不同的单词进行编号
然后每个单词的词向量为长度为 n 的向量，该单词编号下标位置值为 1，其余位置全为 0

Bag-of-word 表示

对句子/文档进行编码，过程:
首先对 n 个不同的单词进行编号
然后每个文档编码为长度为 n 的向量，每个位置的值为该下标编号的单词在文档中出现的次数

Word Embedding 表示

训练所得的单词的连续的特征表示，可用于衡量词间相似度



Embedding matrix is a learned parameter of the model 参数输入

有哪些方法，网络结构什么样，举一些例子，优缺点

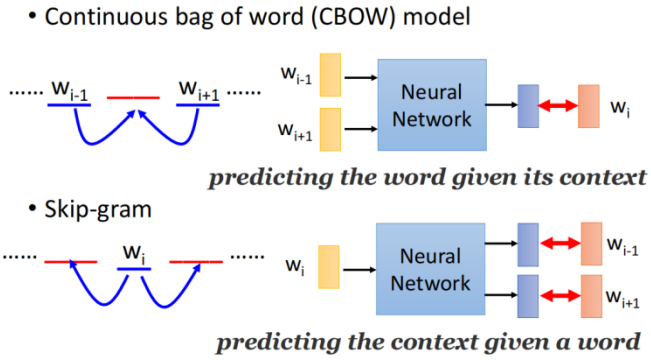
传统方法不能表示词间相似性，深度学习方法能够将语义相似的单词映射到相似的特征空间

1.基于预测的词嵌入（word-embedding）模型

输入 one-hot 编码映射到特征空间，使得语义相似的单词映射到相似的特征空间

CBOW 模型：

CBOW的基本思想是，给定一个单词的上下文（即窗口内的其他单词），预测该单词本身。例如，对于句子“The cat climbed up the tree”，如果窗口大小为5，那么当中心单词为“climbed”时，上下文单词为“The”、“cat”、“up”和“the”。CBOW模型要求根据这四个上下文单词，计算出“climbed”的概率分布。



Skip-gram

Skip-Gram 是给定 input word 来预测上下文

CBOW 模型相比于传统的基于计数或者基于矩阵分解等方法生成词向量有以下优点：

能够**利用大规模语料库进行训练**

能够**学习到高质量且低维度**（通常在 50~300 之间） **的稠密向量**，节省存储空间和计算资源

能够**捕捉单词之间的复杂关系**，如同义词、反义词、类比关系等

CBOW 模型也有以下缺点：

忽略了上下文单词的顺序，只考虑了它们的累加效果（Solve：Positional Encoding）

对于**低频或者生僻单词，可能无法生成准确的词向量**

需要**大量的训练时间和内存空间**

总结来说就是CBOW模型中input是context（周围词）而output是中心词，训练过程中其实是在从output的loss学习周围词的信息也就是embedding，但是在中间层是average的，一共预测V(vocab size)次就够了。skipgram是用中心词预测周围词，预测的时候是一对word pair，等于对每一个中心词都有K个词作为output，对于一个词的预测有K次，所以能够更有效的从context中学习信息，但是总共预测K*V词。

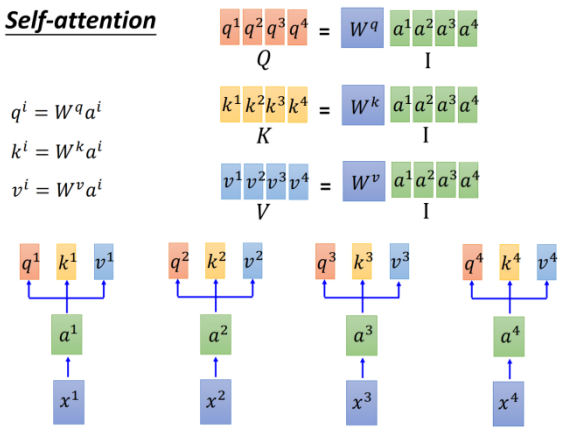
因此，skip gram的训练时间更长，但是对于一些出现频率不高的词，在CBOW中的学习效果就不日skipgram。反正mikolov自己说的skip准确率比CBOW高。。。

六、预训练（深度二）

三种方式，范式，举一些例子，代表性工作，优缺点

回顾：Transformer

Self-Attention 模块的引入：替换不能并行化的 RNN



自注意力只 Q、K、V 都对相同的 A 乘以权重矩阵变换而来

Self-attention

拿每个 query q 去对每个 key k 做 attention

d is the dim of q and k

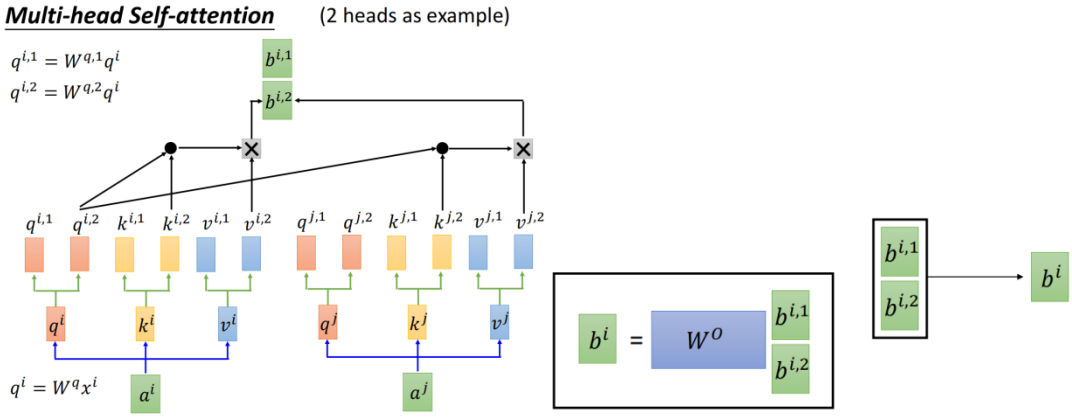
Scaled Dot-Product Attention: $\alpha_{1,i} = q^1 \cdot k^i / \sqrt{d}$

dot product

$$\hat{\alpha}_{1,i} = exp(\alpha_{1,i}) / \sum_j exp(\alpha_{1,j}) \quad b^1 = \sum_i \hat{\alpha}_{1,i} v^i$$

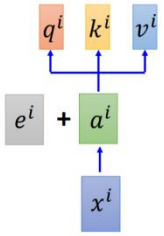
（B_i：Q_i 的自注意力结果）

多头自注意力：每个头各自跑各自的，各个头最后得到的结果再加权得到最终结果



位置编码：三角函数周期性编码

Positional Encoding



- No position information in self-attention.
- Original paper: each position has a unique positional vector e^i (not learned from data)

$$\overrightarrow{p_t}^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k . t), & \text{if } i = 2k \\ \cos(\omega_k . t), & \text{if } i = 2k+1 \end{cases}$$

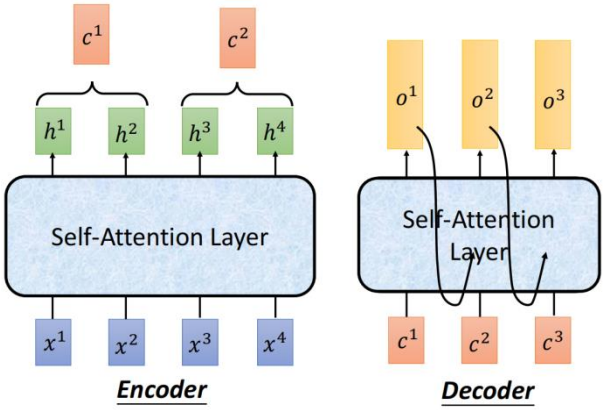
$$\omega_k = \frac{1}{10000^{2k/d}}$$

t is the desired position in an input sentence

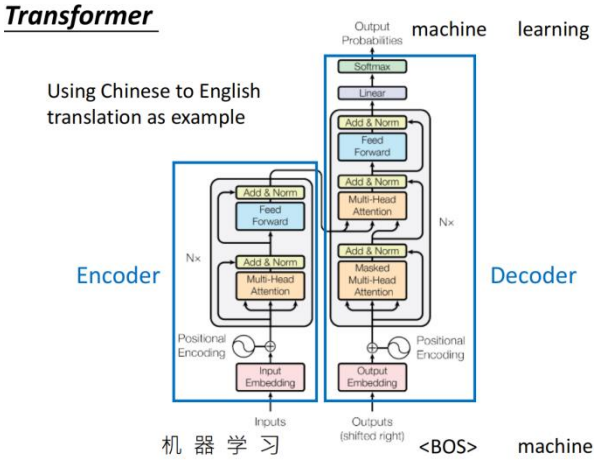
$$\overrightarrow{p_t} = \begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \vdots \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_{d \times 1}$$

Transformer 的 Encoder：并行生成
Decoder：一个一个生成

Seq2seq with Attention



Transformer 架构：



模型预训练的三种方式：
1.Decoders(GPT)

2.Encoders(Bert)
3.Encoder-Decoders(BART)

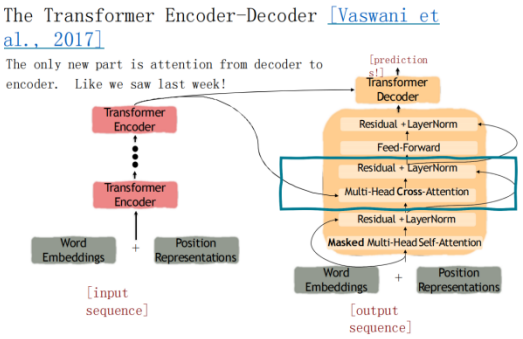
1、子词模型（subword models）：Byte-Pair Encoding (字节对编码，BPE)

2、Motivating model pretraining from word embeddings
之前的 Embedding 没有关注上下文，那么如下例子：

Consider *I* ***record*** *the* ***record***: the two instances of ***record*** mean different things.

所以，结合上下文训练得到一个单词的 embedding 是十分必要的

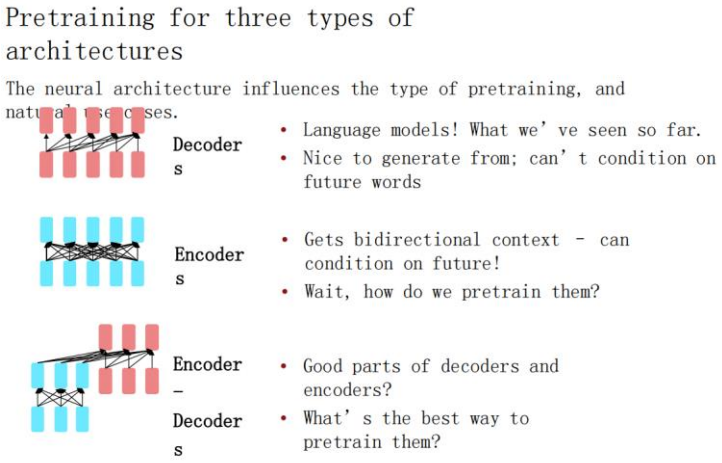
如何进行预训练？
在现代 NLP 中：NLP 网络中的所有（或几乎所有）参数都通过预训练进行初始化。
•预训练方法从模型中隐藏输入的部分，并训练模型重建这些部分。这在构建强模型方面非常有效：对强自然语言处理模型的语言参数初始化的表示。
•从已有语言模型中进行概率分布抽样
概括而言（BERT）：
1、输入数据随机 Mask 部分单词（完形填空）；
2、给定两个句子，预测是否为上下文



预训练的作用：可以通过作为参数初始化（即运用预训练模型训练得到的模型参数作为当前模型的初始化参数）来改进 NLP 应用程序。

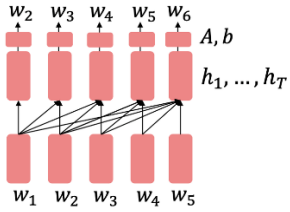
Q：为什么从“训练神经网络”的角度来看，预训练和微调应该有所帮助呢？
A：预训练得到的参数迁移到相似任务，相似任务模型参数对应 Loss 最小的参数取值应与预训练模型参数取值相近，故进行微调。

3、预训练模型的三种方式：
三种方式，范式，举一些例子，代表性工作，优缺点

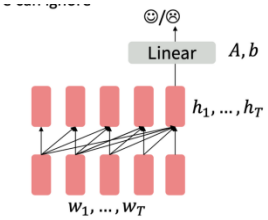


1、预训练 Decoder：

在预训练阶段，使用语言模型(序列前面的词去预测下一个词)，此时的 A 与 b 是从语言模型中学习得到。



在**微调**阶段，可在最后一个词的输出向量后面**接上一个线性层进行分类**，线性层的参数 A、b 不是来自于语言模型，而是在随机产生，在微调中学习得到。



特点：这种预训练方法，擅长文本生成，但是在预训练时像语言模型一样，不能看到未来的词。

典型案例：

GPT

微调中使用 Natural Language Inference，将两句子放在一起（如下）

[START] The man is in the doorway [DELIM] The person is near the door [EXTRACT]

对[EXTRACT]向量加上线性层，输出 **3** 分类(entailing 需要/contradictory 矛盾/neutral 对立)

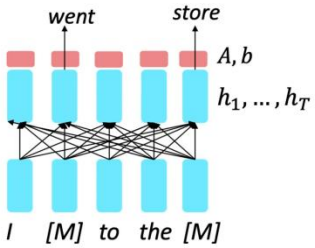
GPT2

更大的数据集，给出指定的文本，使可输出文字

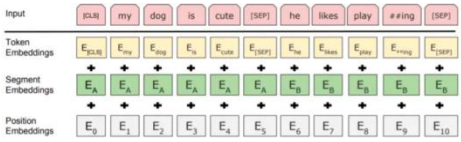
2、预训练 Encoder：

问题：encoder 要求可以使用双向文本，即可以看到未来的词。因为是训练的**双向**上下文语

义信息，这样就不能使用语言模型。
解决：使用 Masked LM(掩码语言模型)：将部分词遮住，去预测被遮住的词。



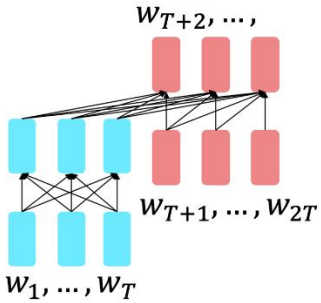
典型案例：
BERT
使用掩码语言模型，需要预测 15%的 token（去掉一些词做完形填空），这 15%的 token 中：
- 80%：用[mask]掩码
- 10%：随机替换成其他的 token
- 10%: token，但是还是需要预测它。目的是**提高预测难度使得能对非掩码词有更强的表达**。
输入：
next sentence prediction：预测输入的两句话是否上下文。实践中，NSP 不是必要的。



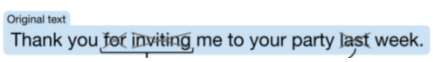
encoders 类的预训练缺点：不擅于文本生成

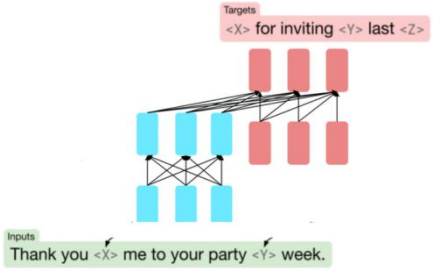
3、预训练 Encoder-Decoder

encoder 部分可以利用双向文本的信息
decoder 部分使用语言模型



典型案例：
T5
将某些词 span 掩住，在 decoder 中预测被掩住的词（这一步有点像语言模型）





它可以被微调来回答问题（属于开放性问答）

BART（双向自回归 Transformer）

BART 的预训练包含两步：

1） 利用任意一种噪声函数分解文本

2） 学习一个模型来重构回原来的文本

预训练学习到了什么？

学到了平凡的事，语法，指代，词江语义/主题，情感，某些原因，一些基础算术（斐波那契数列等）

越来越多的证据表明，预先训练的模型学习了关于语言统计属性的各种东西。

超大模型与上下文学习

GPT3

GPT-3 依旧延续自己的单向语言模型训练方式，只不过这次把模型尺寸增大到了 1750 亿，并且使用 45TB 数据进行训练。

同时，GPT-3 主要聚焦于更通用的 NLP 模型，解决当前 **BERT 类模型的两个缺点：**

1. 对于**领域内有标签数据的过分依赖**：虽然有了预训练+精调的两段式框架，但还是少不了一定量的领域标注数据，否则很难取得不错的效果，而标注数据的成本又是很高的。
2. 对于**领域数据分布的过拟合**：在精调阶段，因为领域数据有限，模型只能拟合训练数据分布，如果数据较少的话就可能造成过拟合，致使模型的泛化能力下降，更加无法应用到其他领域。

因此 **GPT-3 的主要目标是用更少的领域数据、且不经过精调步骤去解决问题。**

Zero-shot、One-shot、Few-shot 都是完全不需要精调的，区别在于给出的 examples 是多还是少。

Zero-shot：没有给出例子

one-shot：给出一个例子

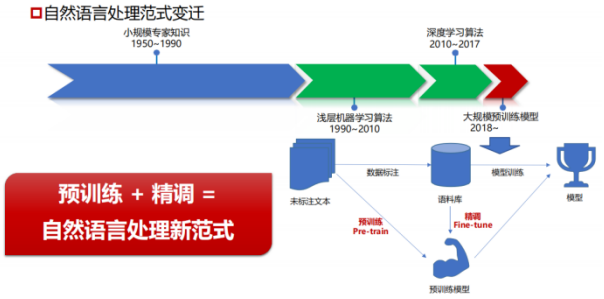
Few-shot：给出多个例子

七、开放性题目（深度三）

介绍一个先进的技术、谈谈看法、发展、展望

概念性的东西

□ChatGPT是2022年11月美国人工智能公司OpenAI所推出的**生成式对话预训练模型**。它通过对话的形式进行交互，对话的形式使得其**能够回答后续问题，承认自己的错误，质疑不正确的前提，并拒绝不适当的请求**。



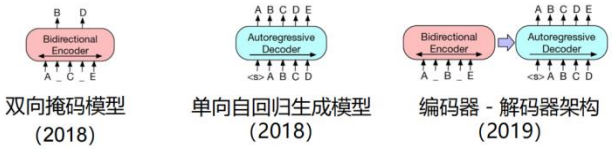
□两种任务类型

□语言模型

□ 通过历史词序列预测**下一个词**

□完形填空

□ 通过周围的词预测**中间的词**



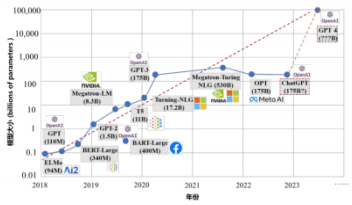
双向掩码模型 (2018)　单向自回归生成模型 (2018)　编码器 - 解码器架构 (2019)

从左到右是：BERT、GPT、BART

预训练语言模型成为自然语言处理领域全新的技术范式

□模型规模与表现**正相关**，因此不停追求越来越大的规模

□随着模型规模越来越大，“**涌现**”出了令人惊讶的“智能”



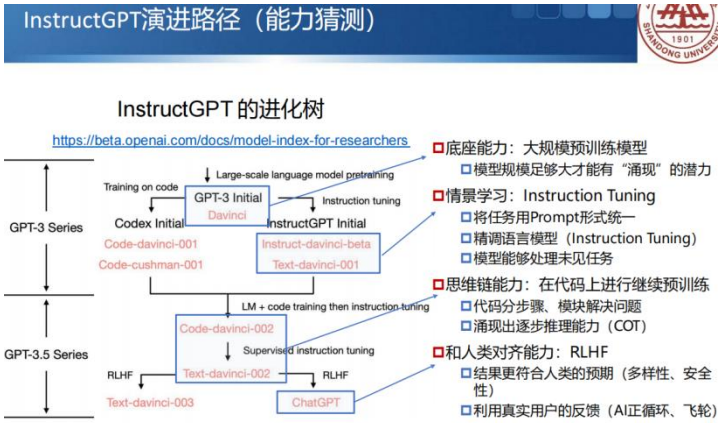
通过什么途径激发他的能力？情境学习，指令学习，思维链。



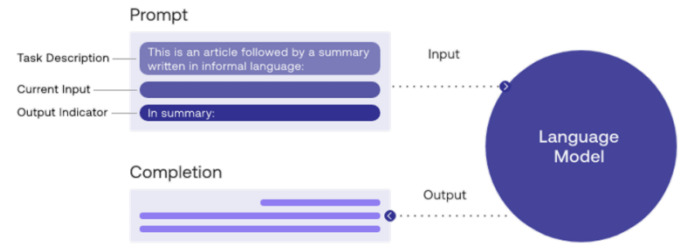
大模型的涌现能力 改变传统学习范式　大模型的涌现能力 打破模型参数约束　人在环路增强 对齐人类意图

ChatGPT的三个关键能力

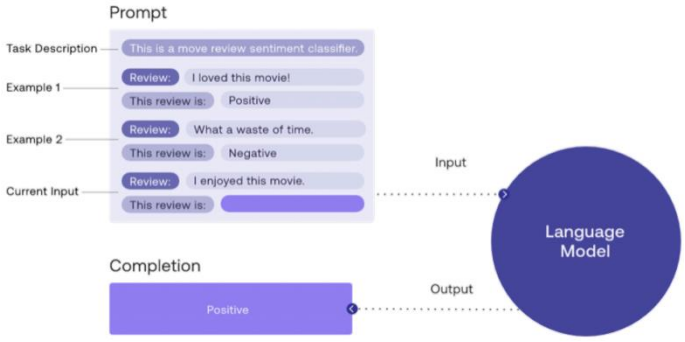
***



第一部分：情境学习的**指令微调**详解：就是告诉模型要**完成什么样的任务、输入格式是什么、输出格式是什么**。这些称为 Prompt（提示词）。

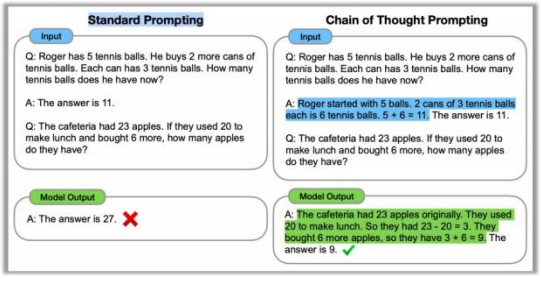


下面这个是给出几个例子，让模型理解怎么回答这类问题。再给出一个新的类似问题，模型可以回答。

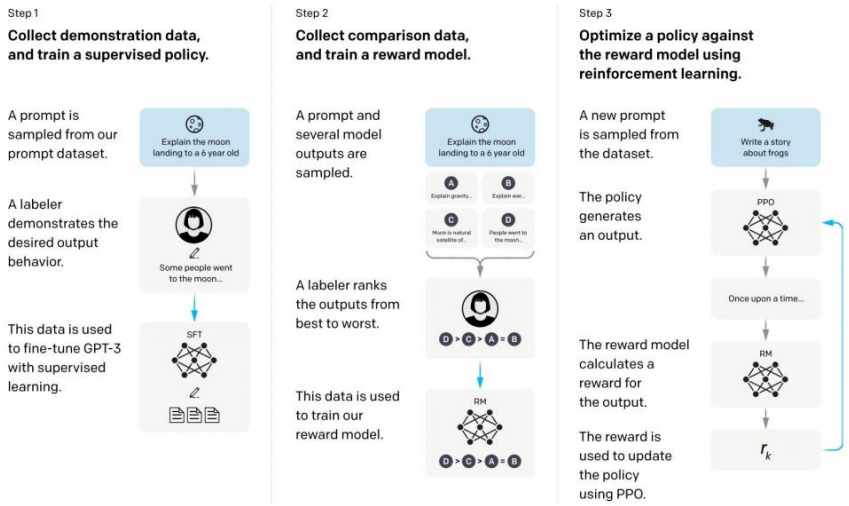


第二部分：思维链

思维链表示一系列中间推理步骤，相当于在求解问题过程中将解题步骤也写出来



第三部分：指令学习-强化学习-人在环路



第一步：收集数据并进行有监督学习。采样一个提示词，由人给出回答，GPT3 根据输入输出进行微调。

第二步：收集比较数据训练奖励模型。采样一个提示词并给出多种不同的回答，由人去对回答的好坏排序，用于不同的奖励值来训练模型。

第三步：利用强化学习优化策略反奖励模型。从数据集中取样的新提示符，模型生成一个输出，奖励模型计算针对输出的奖励，用于更新大模型本身。

OPENAI 训练 GPT 的过程，第一步第二步



SFT Data

split	source	size
train	labeler	11,295
train	customer	1,430
valid	labeler	1,550
valid	customer	103

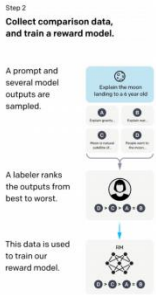
Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

第二步：训练奖励模型

- 收集排序数据，训练奖励模型
  - 采样出一条 prompt 以及第一步模型的多条输出
  - 标注人员对模型的输出进行由好到坏的排序
  - 奖励模型由参数量为**6B的SFT模型初始化**，输入prompt以及第一阶段模型的回复，输出是0-1之间的分数。利用排序好的数据，根据Pairwise Ranking Loss优化奖励模型来模拟标注人员的偏好

RM Data

split	source	size
train	labeler	6,623
train	customer	26,584
valid	labeler	3,488
valid	customer	14,399

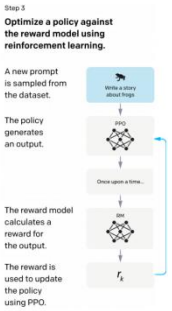


第三步：强化学习

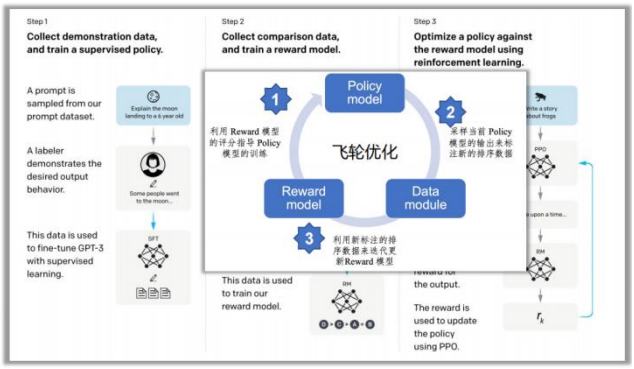
- 使用强化学习PPO算法优化policy
  - 从数据集中采样出一条新的prompt
  - Policy模型首先利用第一阶段微调得到的SFT模型初始化，然后根据prompt生成对应的模型输出
  - 第二步训练得到的奖励模型对该输出计算reward，并利用该reward通过proximal policy optimization (PPO) 算法优化Policy

PPO Data

split	source	size
train	customer	31,144
valid	customer	16,185



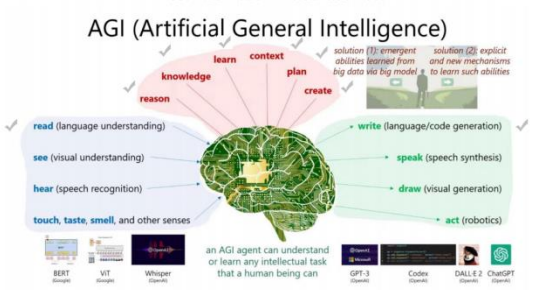
第四步：飞轮优化



未来展望

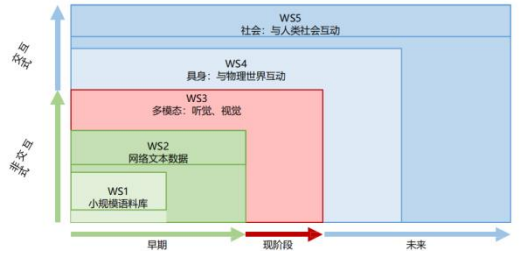
点亮更多的技能

ChatGPT将加速通用人工智能的实现。



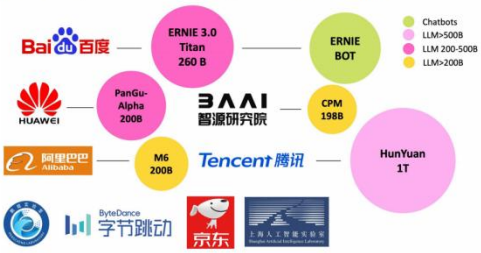
扩大交互范围

- 模型将继续沿着“**同质化**”和“**规模化**”的道路发展
- 拓展除语言之外的认知能力，寻找新的“**知识**”来源
  - 规则 → 算法 → 数据 → **体验**（Experience）
  - Bisk等人（2020）将其称为“**世界范围**”（World Scope，WS）



我们如何应对ChatGPT的挑战?

- 自主可控
  - 联合企业或超算中心，训练自己的大模型
  - 以开源大模型（OPT、BLOOM等）为基础继续预训练



统计学习重点四部分：（大部分出题都在这四块里）
语言模型
自动分词基本算法
统计机器翻译和神经机器翻译
情感分析
深度学习重点三部分：
Word to vec（有哪些方法，网络结构什么样，举一些例子，优缺点）
预训练（三种方式，范式，举一些例子，代表性工作，优缺点）
大模型（概念性的东西，激发他的能力，情境学习，指令学习，serve near，OPENAI 训练 GPT 的过程，第一步第二步）

今年没说一、文本摘要

介绍文本摘要的方法并解释

今年没说二、迁移学习（没提到）

源数据有标签，目标数学没有标签

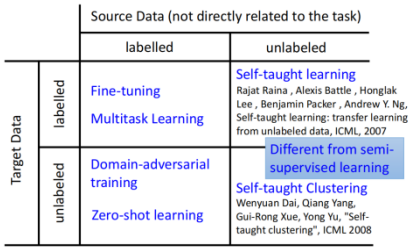
下面是之前深度学习的：

Transfer Learning

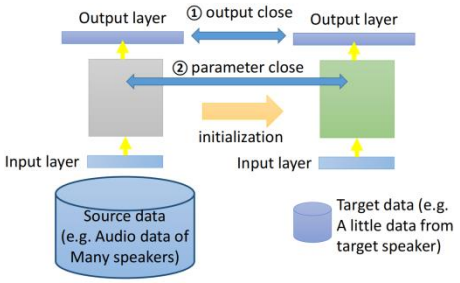
Source data 与 Target data、labeled 与 unlabeled 的区别：P546

Source 与 target 数据量等方面的差异(P547)
Source data 数据量大，并且是广泛的数据
target data 数据量小，是聚焦的数据

Transfer Learning - Overview



Conservative(保守) Training



对于语音，冻结输出层和中间参数，微调输入层

Layer transfer 的功用

复制一些层的参数，仅对其余层进行训练（防止过拟合）
微调整个网络（如果有足够的数据）

Layer transfer 在语音与图像上的区别：
语音取最后几层，图像取前几层；
因为浅层为细节特征，深层为语义特征。更高的层次学习更高级的表示。对于 NLP，语义是通用的。对于 CV，视觉特征是通用的。